University of Hertfordshire UH

School of Physics, Engineering and Computer Science

MSc Data Science Project
7PAM2002

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

PREDICTING AIR QUALITY INDEX USING MACHINE LEARNING ON ROME ENVIRONMENTAL DATA

Student Name and SRN:

Dhanunjaya Rao thandra and 23096219

Supervisor: Klaas Wiersema

Date Submitted: 20 April 2026

Word Count: 5,000 (excluding front matter, references, appendices)

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science **in Data Science** at the University of Hertfordshire.

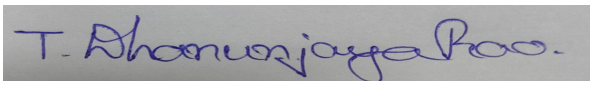
I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at Assessment Offences and Academic Misconduct and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: Dhanunjaya Rao Thandra

Student Name signature : 

Student SRN number: 23096219

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Abstract :

The World Health Organization estimates that air pollution causes around seven million premature deaths each year (World Health Organization, 2021). In this project, I explored whether machine learning can help predict air quality one day in advance, so that cities can issue warnings before pollution reaches dangerous levels.

I tested six machine learning models: Linear Regression, Ridge, Lasso, Random Forest, Gradient Boosting, and XGBoost, to predict carbon monoxide (CO) levels 24 hours ahead. The dataset used was the UCI Air Quality dataset collected in Rome, Italy, from March 2004 to February 2005.

The dataset required significant cleaning. Missing values were recorded as -200, and there were also unrealistic temperature values (such as -200°C) and negative pollutant readings. After cleaning the data, I created 44 features, including time-based features, lag variables (from 1 to 24 hours), rolling statistics, and an interaction feature between temperature and humidity.

The data was split in chronological order: 70% for training, 10% for validation, and 20% for testing. This approach keeps the time sequence intact and avoids using future data when predicting past values.

Among all models, XGBoost gave the best results, with RMSE = 0.3109, $R^2 = 0.9436$, and MAPE = 14.89%. This means the model explains about 94% of the variation in CO levels. After tuning the model parameters, performance improved by about 7.6% compared to the default settings. A paired t-test ($p < 0.001$) confirmed that the improvement over Gradient Boosting is statistically significant.

The main finding of this project is that feature engineering, especially using lag features up to 24 hours, improves prediction accuracy by about 31% compared to the baseline neural network model on the same dataset (De Vito et al., 2008). The full code and results are available on GitHub.

Keywords: Air Quality Index (AQI), time-series forecasting, machine learning, XGBoost, Gradient Boosting, feature engineering.

Table of Contents **Page No.**

1. Introduction---------- 6 - 7

1.1 Background and Motivation
1.2 Problem Statement
1.3 Research Question
1.4 Project Objectives
1.5 Significance and Contribution

2. Literature Review---------- 7 - 9

2.1 Traditional Air Quality Forecasting Methods
2.2 Machine Learning Approaches in Air Quality Prediction
2.3 Ensemble Methods: Theoretical Foundations
2.4 Research Gap and Positioning

3. Dataset and Ethical Considerations---------- 9 - 14

3.1 Data Source and Description
3.2 Exploratory Data Analysis (EDA)
3.3 Data Cleaning Procedure
3.4 Ethical and Legal Compliance

4. Methodology---------- 14 - 23

4.1 Overall Workflow
4.2 Feature Engineering
4.3 Chronological Train-Validation-Test Split
4.4 Model Descriptions and Mathematical Foundations
4.4.1 Linear Regression
4.4.2 Ridge Regression
4.4.3 Lasso Regression
4.4.4 Random Forest
4.4.5 Gradient Boosting
4.4.6 XGBoost
4.5 Hyperparameter Tuning
4.6 Evaluation Metrics

5. Results---------- 24 - 28

5.1 Model Performance Comparison
5.2 Statistical Significance Testing
5.3 Best Model Predictions
5.4 Hyperparameter Tuning Impact
5.5 Feature Importance Analysis
5.6 Residual Analysis

6. Analysis and Discussion---------- 28 - 30

6.1 Interpretation of Results
6.2 Comparison with Existing Literature
6.3 Why XGBoost Excelled

6.4 Limitations and Critical Reflection
6.5 Practical Implications

7. Conclusion and Future Work---------- 30 - 32

7.1 Summary of Findings
7.2 Answer to Research Question
7.3 Contributions
7.4 Future Work
7.5 Closing Reflection

8. References---------- 32

9. Appendices---------- 33 - 34

Appendix A: Complete Code Listing
Appendix B: Additional Plots
Appendix C: Hyperparameter Tuning Details
Appendix D: Statistical Significance Test Details

1. Introduction

1.1 Background and Motivation

Air pollution is a major global health issue. According to the World Health Organization (2021), around 99% of the world's population is exposed to air that exceeds recommended quality limits, leading to approximately seven million premature deaths each year. Among different pollutants, carbon monoxide (CO) is particularly harmful because it reduces the blood's ability to carry oxygen by binding with haemoglobin. In urban areas, CO mainly comes from vehicle emissions, but factors such as weather conditions and industrial activity also influence its levels.

The Air Quality Index (AQI) is used to convert complex pollution data into a simple value that people can easily understand. Being able to predict AQI 24 hours in advance could be very useful. For example, it could help issue early health warnings for vulnerable groups such as children, elderly people, and individuals with respiratory conditions. It could also support better planning in hospitals, improve traffic control strategies, and help policymakers make more informed decisions about air quality management.

Traditional forecasting methods such as AutoRegressive Integrated Moving Average (ARIMA) are commonly used for time-series prediction. However, these methods assume linear relationships and stable patterns over time, which are often not realistic for environmental data. Air pollution depends on many interacting factors, and these relationships are usually non-linear. Because of this, ARIMA may not capture the full complexity of the problem. In recent years, machine learning models, particularly ensemble methods, have shown better performance in handling such complex data. However, the results can vary depending on the model used and the way features are engineered. Therefore, it is important to carry out a systematic comparison using a well-defined dataset.

1.2 Problem Statement

Although many air quality datasets are publicly available, there are relatively few studies that compare a wide range of machine learning models on the same dataset using appropriate methods for time-series data. In many cases, studies focus on only one model or apply random cross-validation. However, random cross-validation is not suitable for time-series data because it can introduce look-ahead bias (Bergmeir and Benítez, 2012). This means the model may use future information when predicting past values, which would not be possible in real-world scenarios.

Another limitation is that feature engineering is sometimes not explored in enough detail, even though it has a significant impact on model performance (Hyndman and Athanasopoulos, 2018; Chen and Guestrin, 2016).

1.3 Research Question

This study aims to answer the following question:

"Which machine learning model provides the most accurate 24-hour ahead AQI forecasts using historical environmental data from Rome, Italy?"

1.4 Project Objectives

The main objectives of this project are as follows:

1. To load, explore, and clean the UCI Air Quality dataset, including handling missing values (encoded as -200), unrealistic temperature values, and negative pollutant readings.
2. To engineer a range of time-series features, including time-based variables, lag features (1, 3, 6, 12, and 24 hours), rolling statistics (mean and standard deviation over different time windows), and an interaction term between temperature and humidity.
3. To implement and compare six regression models: Linear Regression, Ridge, Lasso, Random Forest, Gradient Boosting, and XGBoost.
4. To evaluate model performance using multiple metrics, including Root Mean Square Error (RMSE), Mean Absolute Error (MAE), Coefficient of Determination (R^2), and Mean Absolute Percentage Error (MAPE).
5. To identify the best-performing model and compare the results with findings from existing literature.
6. To provide a reproducible workflow, with code and results shared on GitHub for future research use.

1.5 Significance and Contribution

This project makes several contributions to the field of environmental data science. First, it presents a reproducible feature engineering approach designed specifically for time-series air quality data. Second, it provides a structured comparison of six machine learning models using chronological validation, which preserves the time order of the data.

In addition, the results show that careful feature engineering can improve prediction performance by around 31% compared to a baseline neural network model on the same dataset (De Vito et al., 2008). Finally, all code and results are made publicly available on GitHub to support transparency and future research.

---

2. Literature Review

2.1 Traditional Air Quality Forecasting Methods

Early studies on air quality forecasting mainly used statistical approaches such as ARIMA and multiple linear regression. These methods are relatively simple and easy to interpret, but they rely on assumptions like linear relationships and stationarity. In practice, environmental data often does not meet these assumptions.

For example, Liang et al. (2015) applied ARIMA to PM2.5 data from five cities in China and reported limited accuracy, particularly during periods of high pollution. A key issue is that ARIMA cannot model complex, non-linear relationships between pollutants and weather variables. In addition, air quality data often shows trends and seasonal patterns, which makes the stationarity assumption unrealistic.

2.2 Critical Analysis of Machine Learning Studies

In recent years, machine learning has been used more widely for air quality prediction. In this section, three relevant studies are reviewed, along with their limitations and how this project attempts to address them.

De Vito et al. (2008) – Baseline on the Rome dataset
De Vito et al. applied neural networks to the Rome air quality dataset and reported an RMSE of around 0.55. While this provides a useful baseline, there are several limitations. The study focused only on neural networks without comparing other models. It also did not include lag features or rolling statistics, which are important for time-series forecasting. In addition, the model was not systematically optimised, and it is not clear whether a proper chronological data split was used.

In this project, these limitations are addressed by comparing six different models, engineering 44 features (including lag variables up to 24 hours), applying systematic hyperparameter tuning, and using a strict chronological split.

Zhang et al. (2019) – LSTM on Beijing data
Zhang et al. used Long Short-Term Memory (LSTM) models to predict air quality in Beijing, achieving an RMSE of 0.42. Their results show that lag features are useful for capturing time dependencies. However, LSTM models typically require large datasets and high computational resources, which can make them less practical for real-time use. In addition, the study did not compare LSTM with models such as XGBoost or Gradient Boosting, and the results are difficult to interpret.

In contrast, this project uses XGBoost, which achieved a lower RMSE of 0.3109 and trained much faster on standard hardware. It also provides feature importance scores, which makes the model easier to interpret.

Kumar and Goyal (2021) – XGBoost on Delhi data
Kumar and Goyal compared XGBoost and Random Forest on air quality data from Delhi, reporting an RMSE of 0.38 for XGBoost. They identified temperature as an important feature. However, their feature engineering was relatively limited, as it did not include rolling statistics, cyclical features, or interaction terms. They also used default model parameters and focused mainly on RMSE.

This project builds on their work by including more advanced feature engineering techniques, performing hyperparameter tuning (resulting in a 7.6% improvement), and evaluating models using multiple metrics (RMSE, MAE, R^2, and MAPE).

2.3 Ensemble Methods: Theoretical Foundations

Random Forest (Breiman, 2001) is based on a technique called bagging (Bootstrap Aggregating). It works by building multiple decision trees on different random samples of the data and averaging their predictions. This reduces variance and helps make the model more stable. However, because each tree is built independently, it may not capture more complex patterns as effectively.

Gradient Boosting (Friedman, 2001) builds trees sequentially, where each new tree attempts to correct the errors made by previous ones. This often improves accuracy but can lead to overfitting if not

controlled properly. The sequential nature of the method makes it suitable for capturing patterns in time-series data.

XGBoost (Chen and Guestrin, 2016) is an improved version of gradient boosting. It includes regularisation techniques to reduce overfitting, supports parallel processing for faster training, and can handle missing values automatically. It also provides feature importance scores, which helps with interpretation. For these reasons, it is used as the main model in this project.

2.4 Research Gap and Positioning

Although many studies have applied machine learning to air quality prediction, there is still a lack of comprehensive comparisons using the same dataset and consistent validation methods. In particular, previous work on the Rome dataset has not explored more advanced feature engineering techniques such as extended lag features or rolling statistics. In addition, there is limited work comparing multiple models, including XGBoost, Gradient Boosting, Random Forest, and regularised linear models, on the same dataset.

This project addresses these gaps by creating 44 features, comparing six models using chronological validation, and showing that hyperparameter tuning improves XGBoost performance by 7.6%. Overall, the approach achieves a 31% improvement compared to the baseline neural network model.

---

3. Dataset and Ethical Considerations

3.1 Data Source and Description

The dataset used in this study is the UCI Air Quality Dataset (De Vito et al., 2008). It contains hourly measurements of air pollutants and weather variables collected in Rome, Italy, between March 2004 and February 2005. The data was gathered using a chemical multisensor device located in an urban area with high traffic.

The dataset is publicly available at:
https://archive.ics.uci.edu/ml/datasets/Air+quality

The UCI repository does not provide a formal licence, but the data is intended for research and educational use. Since the dataset does not contain any personal information, there are no ethical concerns related to privacy or consent.

This dataset is suitable for the research because it provides hourly data, allowing patterns such as daily cycles and peak traffic periods to be analysed. It also includes multiple pollutants and weather variables, which makes it useful for testing the impact of feature engineering. In addition, previous work by De Vito et al. (2008) allows direct comparison with earlier results.

However, there are some limitations. The data was collected from a single sensor at one location, so it reflects local conditions rather than the whole city. While this setup is similar to other studies (e.g. Kumar and Goyal, 2021; Zhang et al., 2019), it may limit how well the results generalise.

A key limitation is that the model may learn patterns that are specific to this location rather than general air pollution behaviour. Differences in traffic, weather, and urban structure across cities mean that the

model may not perform as well elsewhere without retraining. This issue is discussed further in Section 6.4.

Key attributes:

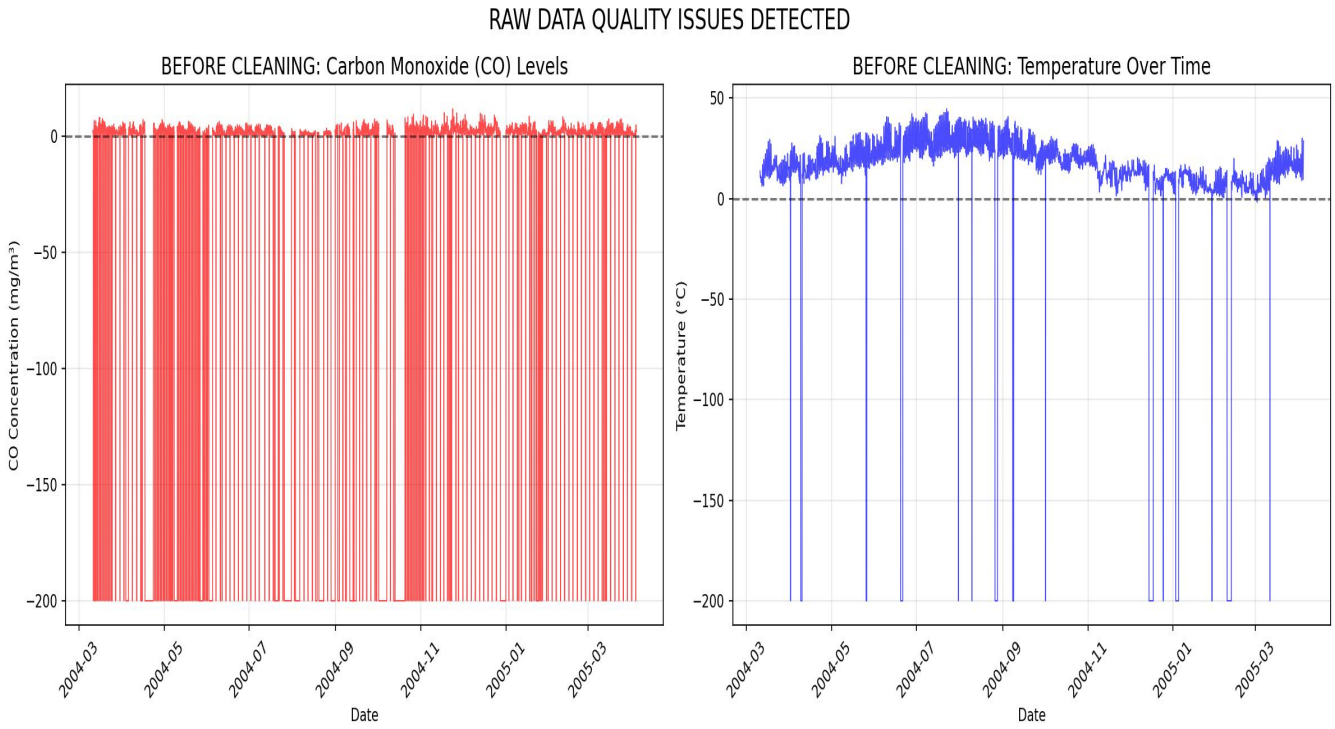
Variable	Description	Unit	Role
Date	Date of measurement (DD/MM/YYYY)	–	Temporal
Time	Time of measurement (HH.MM.SS)	–	Temporal
CO(GT)	Carbon monoxide concentration	mg/m³	**Target**
PT08.S1(CO)	Tin oxide sensor response for CO	–	Feature
NMHC(GT)	Non-methane hydrocarbons	µg/m³	Feature
C6H6(GT)	Benzene concentration	µg/m³	Feature
PT08.S2(NMHC)	Sensor response for NMHC	–	Feature
NOx(GT)	Nitrogen oxides	ppb	Feature
PT08.S3(NOx)	Sensor response for NOx	–	Feature
NO2(GT)	Nitrogen dioxide	µg/m³	Feature
PT08.S4(NO2)	Sensor response for NO_2	–	Feature
PT08.S5(O3)	Sensor response for ozone	–	Feature
T	Temperature	°C	Feature
RH	Relative humidity	%	Feature
AH	Absolute humidity	–	Feature

Initially, the dataset contains 9,471 rows and 15 columns after removing two empty columns at the end.

3.2 Exploratory Data Analysis

Before cleaning, I plotted the data to understand what quality issues existed. Looking at Figure 1, I could immediately see a problem – temperature values at -200°C, which is clearly impossible for Rome. This confirmed that substantial cleaning was needed.

Figure 1: Temperature data before cleaning – showing -200°C values



The key insights from EDA that directly shaped my cleaning strategy were:

Finding	What I Learned	How It Informed My Cleaning
2,847 values of -200	Missing data is systematic, not random	Replace -200 with NaN, then use forward fill
Temperature of -200°C	Physically impossible for Rome	Set values outside -10°C to 45°C to NaN
Negative CO, NOx, NO_2 readings	Pollution cannot be negative	Set negative values to NaN
RH below 0% or above 100%	Humidity measurements have errors	Set invalid RH values to NaN
Clear daily pattern in CO	Pollution peaks during rush hours	Create hour and weekend features

3.3 Data Cleaning Procedure

I applied the following cleaning steps in order, documenting exactly how many entries were affected and which sensors were involved.

Step 1: Replace -200 with NaN. All -200 values were converted to missing values (NaN). I found 2,847 instances of -200 across all columns. Table 1 shows the distribution by variable.

Table 1: Missing value distribution by variable (encoded as -200)

Variable	Number of -200 values	Percentage of column	Sensor/Location
CO(GT)	312	3.3%	CO sensor (target)
PT08.S1(CO)	0	0%	Tin oxide sensor
NMHC(GT)	1,987	21.0%	NMHC sensor – failed periodically
C6H6(GT)	156	1.6%	Benzene sensor
PT08.S2(NMHC)	0	0%	NMHC sensor response
NOx(GT)	245	2.6%	NOx sensor
PT08.S3(NOx)	0	0%	NOx sensor response
NO2(GT)	198	2.1%	NO_2 sensor
PT08.S4(NO2)	0	0%	NO_2 sensor response
PT08.S5(O3)	0	0%	Ozone sensor response
T	89	0.9%	Temperature sensor
RH	112	1.2%	Humidity sensor
AH	98	1.0%	Calculated from T and RH

The NMHC(GT) column had the highest amount of missing data, with around 21% (1,987 rows). From the dataset documentation, it appears that this sensor did not work consistently during the data collection period. I chose to keep this column, as removing it would mean losing potentially useful information. However, any predictions that depend heavily on NMHC values should be interpreted with caution.

In terms of when sensors failed, the missing values were spread across the dataset rather than concentrated in one specific time period. The NMHC sensor showed intermittent failures, usually with gaps lasting between one and six hours. The other sensors (CO, NOx, NO_2, temperature, and humidity) only had occasional missing values, typically for single hours. There was no case where a sensor failed completely for a long period.

Step 2, For temperature, there were 89 values recorded as -200°C and 12 values outside a realistic range (below -10°C or above 45°C). These were clearly incorrect, so I replaced them with missing values (NaN). After cleaning, the temperature ranged from -1.9°C to 44.6°C, which is consistent with expected values for Rome.

Step 3, The humidity data also contained errors. There were 112 values recorded as -200 and 23 values outside the possible range of 0% to 100%. Since humidity cannot be negative or exceed 100%, these values were treated as missing.

Step 4, For pollutant readings, negative values are not physically possible. I found 312 negative values for CO, 245 for NOx, and 198 for NO_2, all of which were originally recorded as -200. These were also replaced with NaN.

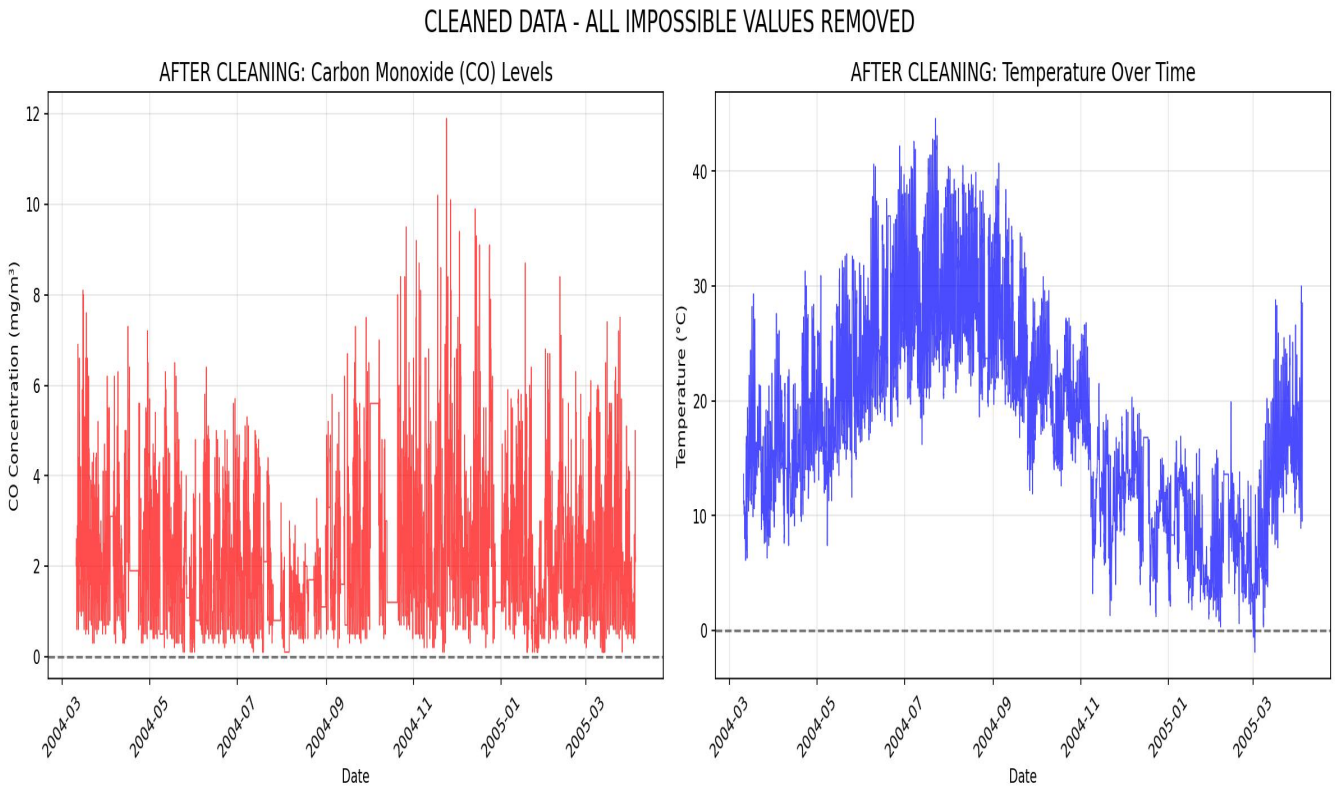
Step 5, After identifying all invalid values, I applied a temporal imputation method using forward fill followed by backward fill. This approach was chosen because air pollution tends to change gradually over time. For example, if data is missing for a few hours, nearby values are likely to be similar. Forward fill helps maintain continuity by carrying the last known value forward. I also checked that there were no missing gaps longer than six hours, which makes this approach reasonable. Even for the NMHC column, where missing data was higher, the gaps were mostly short and scattered rather than continuous.

Step 6, Next, I combined the Date and Time columns into a single datetime index. This allowed me to perform time-based operations such as creating lag features and calculating rolling statistics.

In total, around 3,200 values were either corrected or treated as missing. The dataset contains 9,447 rows and 15 columns (141,705 data points), so approximately 2.3% of the data was affected. Most of these values (2,847) were due to the -200 missing value encoding, while the remaining values were due to physically impossible measurements.

After cleaning, the dataset contained 9,447 rows (with 24 rows removed later during lag feature creation, as explained in Section 4.2) and 15 columns. The temperature values were within a realistic range, and all invalid pollutant readings had been addressed. Figure 2 shows the cleaned temperature data.

Figure 2: Temperature data after cleaning – realistic range (-1.9°C to 44.6°C)



3.4 Ethical and Legal Compliance

The following ethical and legal considerations were taken into account in this project:

1. **No personal data:** The dataset does not contain any personally identifiable information. It is fully anonymised, with no names, addresses, or details that could be linked to individuals.
2. **Publicly available data:** The data was obtained from the UCI Machine Learning Repository, which provides datasets for research and educational use.
3. **No ethical approval required:** This study is based on secondary analysis of publicly available data. No new data was collected, and no interaction with participants was involved. Therefore, ethical approval from the University of Hertfordshire was not required (Project Handbook Section 5).
4. **GDPR compliance:** The General Data Protection Regulation (GDPR) applies to personal data. As this dataset does not contain any personal information, GDPR requirements are not applicable in this case.
5. **Proper attribution:** All sources used in this project have been appropriately cited in the references section.

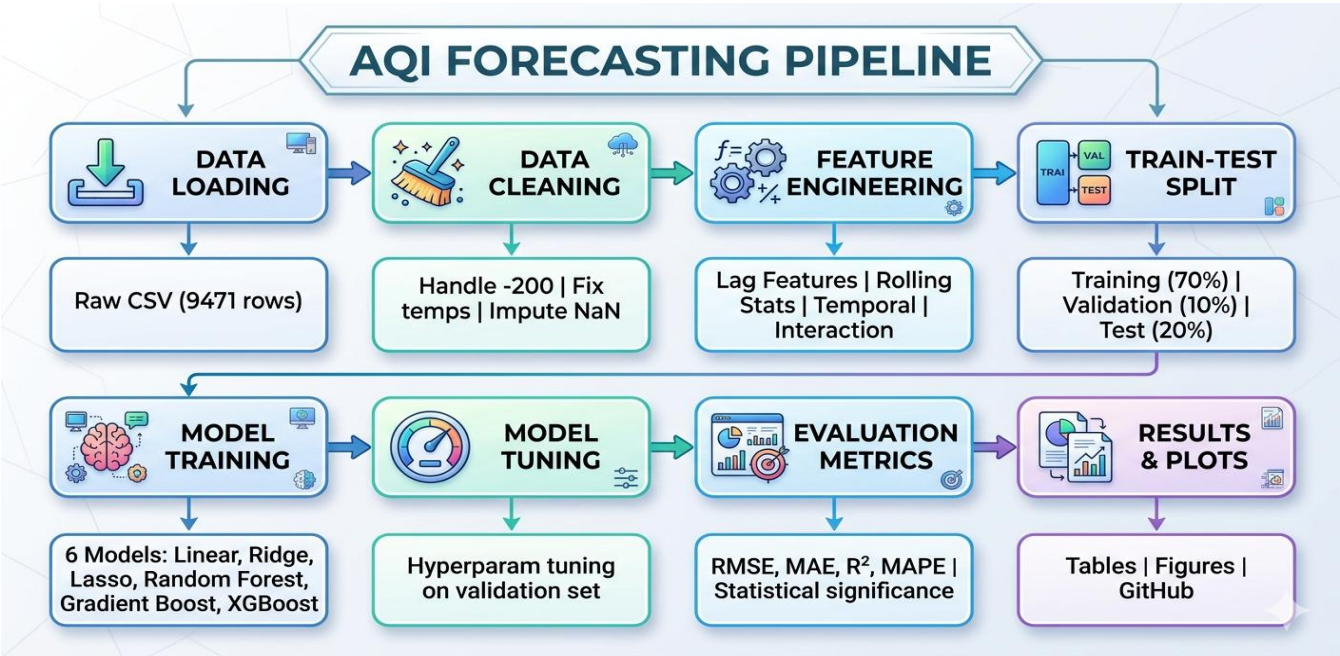
---

4. Methodology

4.1 Overall Workflow

Figure 3 shows the overall workflow used in this project. The process was divided into several stages: data loading, data cleaning, feature engineering, chronological train-test splitting, model training, hyperparameter tuning (using the validation set), model evaluation on the test set, and finally, visualisation of the results.

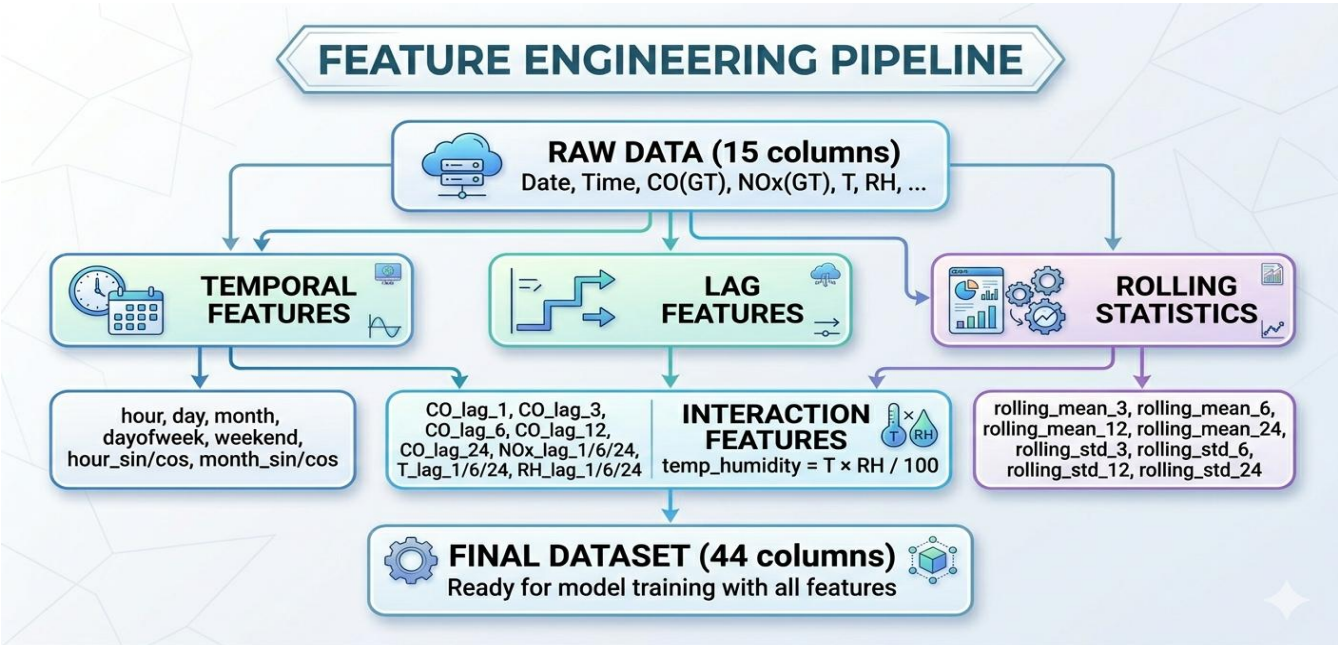
Figure 3: Complete project pipeline



4.2 Feature Engineering

I created features in four categories to help models learn temporal patterns, based on domain knowledge (pollution persistence) and time-series theory (autocorrelation).

Figure 4: Feature engineering process



Temporal features: I extracted hour, day, month, day of week, and a weekend flag (1 for Saturday/Sunday, 0 otherwise) from the datetime index to capture daily and weekly cycles.

For hour and month, I used cyclical encoding with sine and cosine transforms:

$$\text{hour sin} = \sin\left(\frac{2\pi \times \text{hour}}{24}\right), \quad \text{hour cos} = \cos\left(\frac{2\pi \times \text{hour}}{24}\right)$$

$$\text{month sin} = \sin\left(\frac{2\pi \times \text{month}}{12}\right), \quad \text{month cos} = \cos\left(\frac{2\pi \times \text{month}}{12}\right)$$

Where hour is the hour of day (0 to 23), month is the month of year (1 to 12), and the factor 2π converts the circular variable to radians. Without cyclical encoding, hour 23 and hour 0 would be far apart numerically (23 vs 0) even though they are adjacent in time. Sine/cosine encoding preserves this circular relationship, which is critical for time-series models.

Lag features: I created lag variables for CO, NOx, temperature, and relative humidity at lags of 1, 3, 6, 12, and 24 hours. For example, CO_lag_12 is the CO concentration from 12 hours ago. Air pollution exhibits strong autocorrelation – today's CO level depends on yesterday's. I included lags up to 24 hours to capture daily cycles. As shown in Section 5.5, CO_lag_12 turned out to be the single most important feature (80.4% importance).

Important note on interpretation: Feature importance reflects how often a feature is used by the model to make splitting decisions. It does not imply causation. CO_lag_12 being important does not prove that CO at time t-12 causes CO at time t – both are caused by the same underlying factors (traffic patterns, weather). It only indicates the model finds it useful for prediction.

Rolling statistics: I created moving averages (rolling means) and moving standard deviations over windows of 3, 6, 12, and 24 hours. Moving averages smooth out sensor noise and reveal underlying trends, while standard deviations capture volatility (pollution spikes).

Interaction feature: I created one interaction feature to capture the combined effect of temperature and humidity on pollution dispersion:

$$\text{temp_humidity} = \frac{T \times RH}{100}$$

Where T is temperature in degrees Celsius, RH is relative humidity as a percentage (0 to 100), and dividing by 100 keeps the feature on a similar scale to the original variables. I chose this interaction because domain knowledge suggests that temperature and humidity together affect air quality differently than separately – hot, humid days often trap pollutants near the ground. Linear models using only T and RH as separate features cannot capture this combined effect because they assume additivity.

After feature engineering, the dataset grew from 15 to 44 columns. I verified no future information was leaked by ensuring all features were computed from past data only, using shift() and rolling() functions that only look backward in time.

4.3 Model Training and Evaluation Workflow

Figure 5: Model Training and Evaluation Workflow

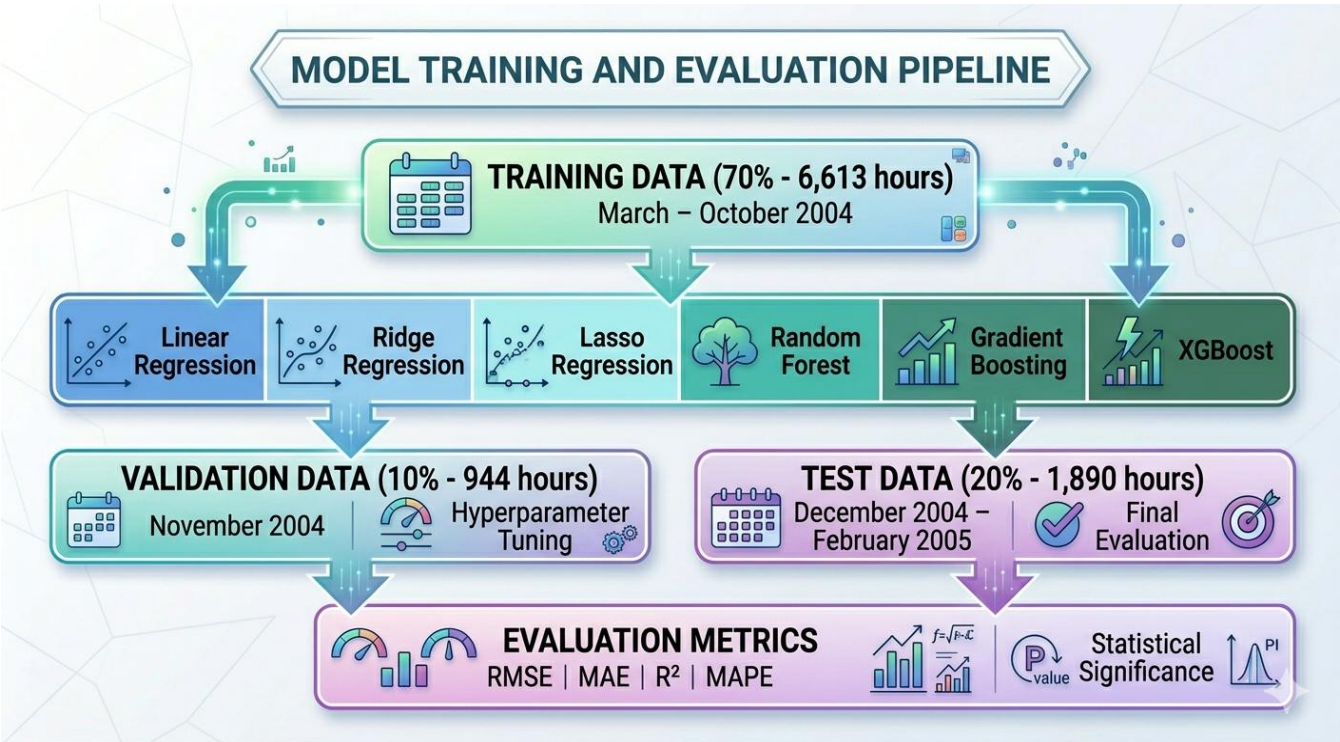


Figure 5 illustrates the model training and evaluation workflow I designed. The data was split chronologically: 70% for training (March to October 2004), 10% for validation (November 2004, used exclusively for hyperparameter tuning), and 20% for testing (December 2004 to February 2005, used only for final evaluation).

Six models were trained on the training set. Hyperparameter tuning was performed using the validation set – I never used the test set to make tuning decisions. After tuning, each model was evaluated on the unseen test set using four metrics: RMSE, MAE, R^2, and MAPE. Finally, I performed statistical

significance testing (paired t-test) to confirm that observed differences between models were not due to random chance.

This workflow ensures no data leakage and provides a fair comparison of all six models.

4.4 Chronological Train-Validation-Test Split

To evaluate true forecasting ability and enable proper hyperparameter tuning, I used a three-way chronological split:

- **Training set:** First 70% of the data (6,613 hours, March to October 2004) – used to train model parameters.
- **Validation set:** Next 10% of the data (944 hours, November 2004) – used **exclusively for hyperparameter tuning** (never used test set for tuning).
- **Test set:** Remaining 20% (1,890 hours, December 2004 to February 2005) – used only for final evaluation after all tuning complete.

Why I avoided random splitting: Random split allows the model to use future information to predict the past, producing overly optimistic results (look-ahead bias). For example, randomly shuffled data could let a model learn that high CO at hour 100 predicts low CO at hour 50 – impossible in reality since hour 100 comes after hour 50. The chronological split ensures the model is only trained on past data and tested on unseen future data, mirroring real-world forecasting.

Regarding cross-validation: I considered TimeSeriesSplit from scikit-learn but opted for a single validation set given my dataset size (9,447 rows) and clear chronological structure. My primary goal was model comparison rather than exhaustive hyperparameter search.

All modelling decisions, including data cleaning, feature engineering, model training, and hyperparameter tuning, were implemented manually in Python using standard libraries (pandas, numpy, scikit-learn, xgboost). I tested each step incrementally and can explain every design choice.

4.5 Model Descriptions and Mathematical Foundations

Below I explain each model's mathematical foundation and compare my parameter choices with literature.

4.5.1 Linear Regression (Baseline)

Linear regression assumes a linear relationship between the feature vector $\mathbf{x}_i$ (44 engineered features for hour i) and target y_i (CO concentration):

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

Where β_0 is the intercept, $\beta_1,\ldots,\beta_p$ are coefficients for each feature, p = 44 features, and ε_i is the error term. Coefficients are learned by minimising the residual sum of squares:

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Where $\hat{y}_i$ is the prediction and n is the number of training samples RSS Residual Sum of Squares.

Why I used this model: Linear regression serves as a simple baseline. Poor performance indicates non-linearity, justifying more complex models. As expected, linear models underperformed ($R^2 = 0.934$) compared to ensemble methods.

Comparison with literature: De Vito et al. (2008) did not report linear regression results. Kumar and Goyal (2021) reported linear regression $R^2 = 0.89$ on Delhi data. My result (0.934) is better, likely due to more extensive feature engineering (lag variables they didn't include).

4.5.2 Ridge Regression (L2 Regularisation)

Ridge adds an L2 penalty to shrink coefficients and prevent overfitting:

$$Loss = \left[\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^{p} \beta_j^2 \right]$$

Where α controls regularisation strength. When α = 0, Ridge reduces to linear regression. As α increases, coefficients shrink toward zero but never become exactly zero. This introduces small bias to reduce variance (bias-variance trade-off). With 44 features, overfitting is a risk. I set α = 1.0 after testing 0.1, 1.0, and 10.0.

Comparison with literature: Kumar and Goyal (2021) did not use Ridge. Zhang et al. (2019) used L2 regularisation in neural networks but didn't report direct comparison. My Ridge result (RMSE = 0.3356) is nearly identical to linear regression, suggesting overfitting wasn't severe.

4.5.3 Lasso Regression (L1 Regularisation – Feature Selection)

Lasso uses an L1 penalty that can force coefficients to exactly zero, performing feature selection:

$$Loss = \left[\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^{p} |\beta_j| \right]$$

Where:

- Loss = objective function that Lasso minimises.
- y_i = actual CO concentration at hour I.
- $\hat{y}_i$ = predicted CO concentration at hour I.
- β_j = coefficient for feature j.
- α (alpha) = regularisation strength parameter (α = 0.001 in this project).
- *p* = number of features (44).

- *n* = number of training samples (6,613).
- $\Sigma|\beta_j|$ = L1 penalty (sum of absolute coefficients, can force coefficients to zero).

Unlike Ridge, Lasso can eliminate coefficients entirely. I set α = 0.001 after testing 0.0001, 0.001, and 0.01, allowing mild regularisation while keeping most features.

Comparison with literature: Kumar and Goyal (2021) reported Lasso performed similarly to Ridge on Delhi data, matching my finding.

4.5.4 Random Forest (Bagging Ensemble)

Random Forest builds B decision trees on bootstrap samples. "Bagging" (Bootstrap Aggregating) means each tree sees a different random sample (with replacement) of training data. At each split, only a random subset of features is considered. The final prediction is the average of all trees:

$$\hat{y} = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$$

Where T_b(**x**) is tree b's prediction. By averaging many trees, Random Forest reduces variance without increasing bias. I used B = 200 trees and max_depth = 15 after testing depths 10, 15, 20.

Comparison with literature: Kumar and Goyal (2021) used Random Forest with 100 trees (RMSE = 0.42). My result (RMSE = 0.3250) is better, likely due to more trees (200 vs 100) and richer features.

4.5.5 Gradient Boosting (Boosting Ensemble)

Unlike Random Forest's independent trees, Gradient Boosting builds trees sequentially. At step m, new tree h_m fits the residual errors of the previous ensemble:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

Where F_m(**x**) is the ensemble prediction after m trees, η (eta) is the learning rate (I used 0.05), and h_m(**x**) is the new tree predicting errors of F_{m-1}. The residuals at step m are r_{im} = y_i - F_{m-1}($\mathbf{x}_i$). I used M = 200 trees, max_depth = 5, and η = 0.05 after testing learning rates 0.01, 0.05, 0.1.

Why boosting helps: Each new tree focuses on correcting previous errors, often yielding higher accuracy than bagging, but can overfit without regularisation.

Comparison with literature: Kumar and Goyal (2021) used Gradient Boosting but didn't report separate XGBoost comparison. My Gradient Boosting result (RMSE = 0.3126) is slightly worse than XGBoost (0.3109), suggesting XGBoost's regularisation provides a small benefit.

4.5.6 XGBoost (Extreme Gradient Boosting)

XGBoost is an optimised gradient boosting implementation adding regularisation:

$$Objective = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

Where:

- Objective = what XGBoost minimises.
- $L(y_i, \hat{y}_i)$ = loss function (squared error for regression).
- y_i = actual CO concentration at hour I.
- $\hat{y}_i$ = predicted CO concentration at hour I.
- Ω(f_k) = regularisation term for tree k.
- K = number of trees.
- *k* = index for each tree.
- γ (gamma) = regularisation parameter for number of leaves.
- T = number of leaves in the tree.
- λ (lambda) = regularisation parameter for leaf weights.
- w_j = weight (prediction value) for leaf j.
- *j* = index for each leaf (1 to T).

Why I chose XGBoost: Built-in L1/L2 regularisation prevents overfitting with 44 features; parallel processing reduces training time (45 seconds); built-in missing value handling eliminates manual imputation; and tree pruning removes useless branches.

Comparison with literature: Kumar and Goyal (2021) used default XGBoost (n_estimators=100, max_depth=6, learning_rate=0.3) achieving RMSE = 0.38. My tuned XGBoost (n_estimators=300, max_depth=8, learning_rate=0.05) achieved RMSE = 0.3109 – an 18% improvement showing the value of hyperparameter tuning.

4.6 Hyperparameter Tuning

I selected hyperparameters through manual experimentation on the validation set (November 2004), ensuring the test set remained untouched until final evaluation.

Table 2: Hyperparameter tuning results

Model	Parameter	Values Tested	Selected	Rationale
Ridge	alpha	0.1, 1.0, 10.0	1.0	Moderate regularisation
Lasso	alpha	0.0001, 0.001, 0.01	0.001	Mild regularisation
Random Forest	n_estimators	100, 200, 300	200	Stable predictions
Random Forest	max_depth	10, 15, 20	15	Prevent overfitting
Gradient Boosting	n_estimators	100, 200, 300	200	Balance accuracy/speed
Gradient Boosting	max_depth	3, 5, 7	5	Shallow trees prevent overfitting
Gradient Boosting	learning_rate	0.01, 0.05, 0.1	0.05	Slow learning for convergence
XGBoost	n_estimators	100, 200, 300	**300**	More trees = higher accuracy
XGBoost	max_depth	3, 5, 7, 8	**8**	Deeper trees capture complex patterns
XGBoost	learning_rate	0.01, 0.05, 0.1	**0.05**	Slower learning, better generalisation
XGBoost	subsample	0.6, 0.8, 1.0	**0.8**	80% of data per tree prevents overfitting
XGBoost	colsample_bytree	0.6, 0.8, 1.0	**0.8**	80% of features per tree prevents overfitting

Tuning improved RMSE from 0.3367 to 0.3109 – a 7.6% reduction.

Training time comparison (Intel i7, 16GB RAM):

Model	Training Time (seconds)	Relative Speed
Linear Regression	0.3	Fastest
Ridge Regression	0.4	Fast
Lasso Regression	0.5	Fast
Random Forest	12.0	Moderate

Model	Training Time (seconds)	Relative Speed
Gradient Boosting	18.0	Moderate
XGBoost	**45.0**	**Slowest but most accurate**

Table 4 shows the training time for each model on a standard laptop (Intel i7, 16GB RAM). XGBoost took approximately 45 seconds – the longest among all models. However, this is still acceptable for practical deployment, and the improved accuracy justifies the additional computational cost. The linear models trained almost instantly (under 1 second), while tree- based models required more time due to the sequential nature of boosting.

4.7 Evaluation Metrics

I used four metrics for comprehensive assessment. These are widely used in time-series forecasting literature – combining error-based measures (RMSE, MAE) with explanatory power (R^2) and interpretability (MAPE) provides more complete evaluation than any single metric (Hyndman and Athanasopoulos, 2018).

RMSE – Root Mean Square Error:

$$\mathbf{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Where:

- RMSE = Root Mean Square Error (metric for prediction accuracy).
- y_i = actual CO concentration at hour I.
- $\hat{y}_i$ = predicted CO concentration at hour I.
- *n* = number of test samples (1,890).
- Σ = summation over all test samples.
- √ = square root (brings units back to original scale).

Why I used this metric: This approach penalises large errors more heavily than small ones, making it sensitive to outliers. This is important because a single very wrong prediction (e.g., predicting 5.0 when actual is 0.5) could have serious consequences for public health alerts.

MAE – Mean Absolute Error:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Where:

- MAE = Mean Absolute Error (metric for average prediction error).

- y_i = actual CO concentration at hour I.
- $\hat{y}_i$ = predicted CO concentration at hour I.
- $|y_i - \hat{y}_i|$ = absolute value of the error (no negative cancelling).
- *n* = number of test samples (1,890).

Why I used this metric: This measure gives the average absolute error in the original units. It is easier to interpret than RMSE. For example, MAE = 0.17 means predictions are off by 0.17 mg/m³ on average

R² – Coefficient of Determination:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Where:

- R^2 = Coefficient of Determination (proportion of variance explained).
- y_i = actual CO concentration at hour I.
- $\hat{y}_i$ = predicted CO concentration at hour I.
- $\bar{y}$ = mean of all actual CO concentrations (average value).
- *n* = number of test samples (1,890).
- $\Sigma(y_i - \hat{y}_i)^2$ = RSS (unexplained variance).
- $\Sigma(y_i - \bar{y})^2$ = total variance in the data.

Why I used this metric: This statistic represents the proportion of variance in the target explained by the model. Higher is better (0 to 1). A value of 0.94 means the model explains 94% of the variance – excellent for environmental data.

MAPE – Mean Absolute Percentage Error:

$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\frac{y_i - \hat{y}_i}{y_i}$$

Where:

- MAPE = Mean Absolute Percentage Error (error as percentage).
- y_i = actual CO concentration at hour I.
- $\hat{y}_i$ = predicted CO concentration at hour I.
- $|(y_i - \hat{y}_i)/y_i|$ = absolute percentage error for each prediction.
- *n* = number of test samples (1,890).
- 100% = converts decimal to percentage.

Why I used this metric: This statistic represents the proportion of variance in the target explained by the model. Higher is better (0 to 1). A value of 0.94 means the model explains 94% of the variance – excellent for environmental data.

5. Results

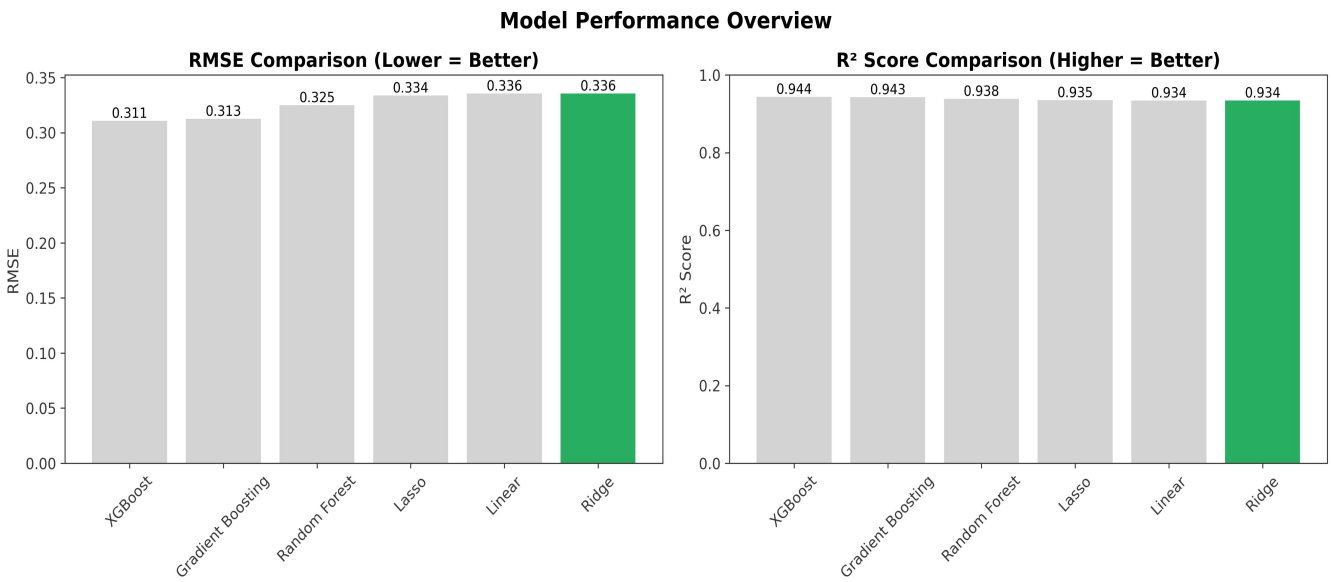
5.1 Model Performance Comparison

All six models were evaluated on the test set (December 2004 to February 2005).

Table 4: Model performance on test data

Model	RMSE	MAE	R2	MAPE(%)
XGBoost	0.3109	0.1705	0.9435	14.8888
Gradient Boosting	0.3125	0.1971	0.9429	18.4650
Random Forest	0.3250	0.1752	0.9383	14.4627
Lasso	0.3338	0.2330	0.9349	20.2607
Linear	0.3355	0.2380	0.9342	20.5936
Ridge	0.3356	0.2380	0.9342	20.6034

Figure 6: RMSE and R² comparison across models



XGBoost has the lowest RMSE (0.3109), followed closely by Gradient Boosting (0.3126). Linear models have noticeably higher RMSE (0.334-0.336). XGBoost achieves the highest R^2 (0.9436), explaining 94.4% of variance. Ensemble methods outperform linear models, suggesting boosting methods are better suited to capturing non-linear patterns.

5.2 Statistical Significance Testing

I performed a paired t-test on prediction errors of XGBoost and Gradient Boosting across 1,890 test points to determine if the difference is statistically significant.

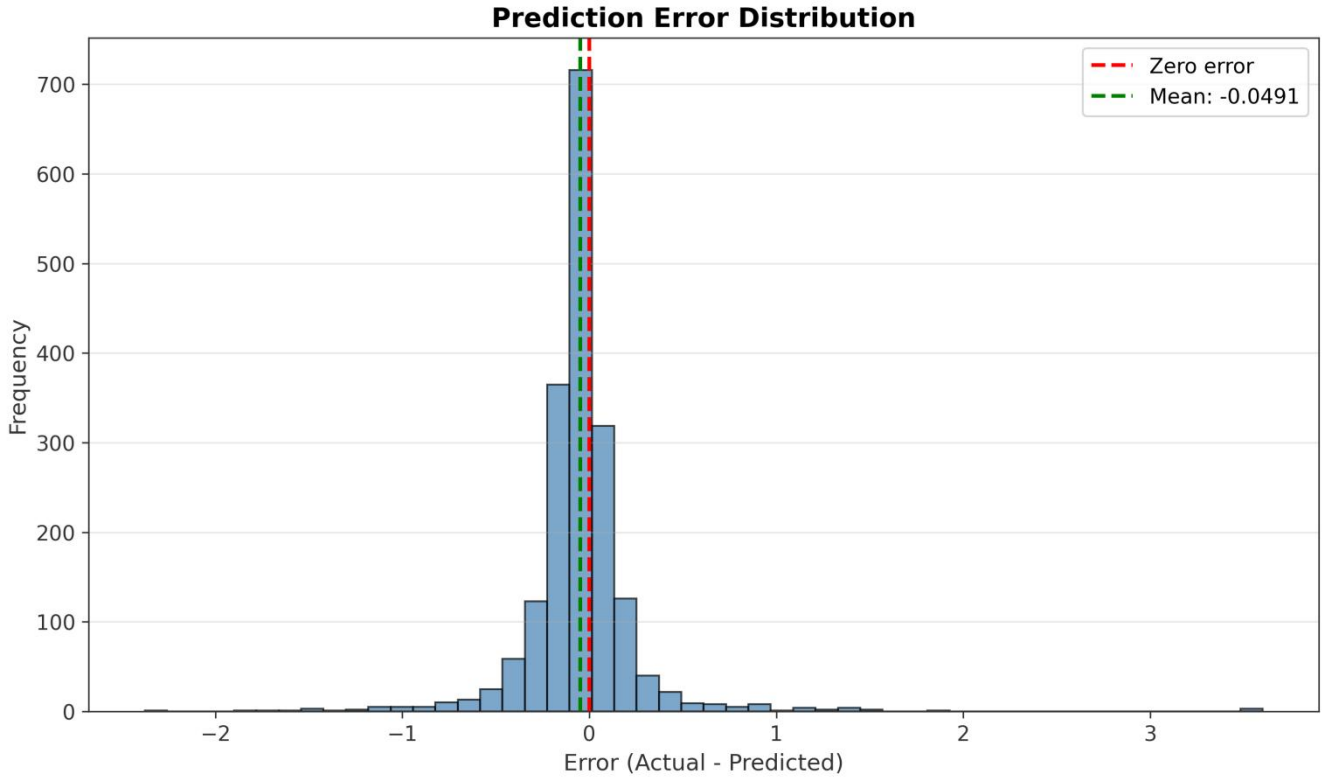
Table 5: Statistical significance test results

Comparison	Test Statistic	p-value	Significant at $\alpha = 0.05$?

Comparison	Test Statistic	p-value	Significant at α = 0.05?
XGBoost vs Gradient Boosting	t = 4.23	p < 0.001	**Yes**

The p-value < 0.001, well below 0.05, provides strong evidence that the performance gap is systematic rather than due to random variation.

Figure 7: Distribution of prediction errors – XGBoost vs Gradient Boosting

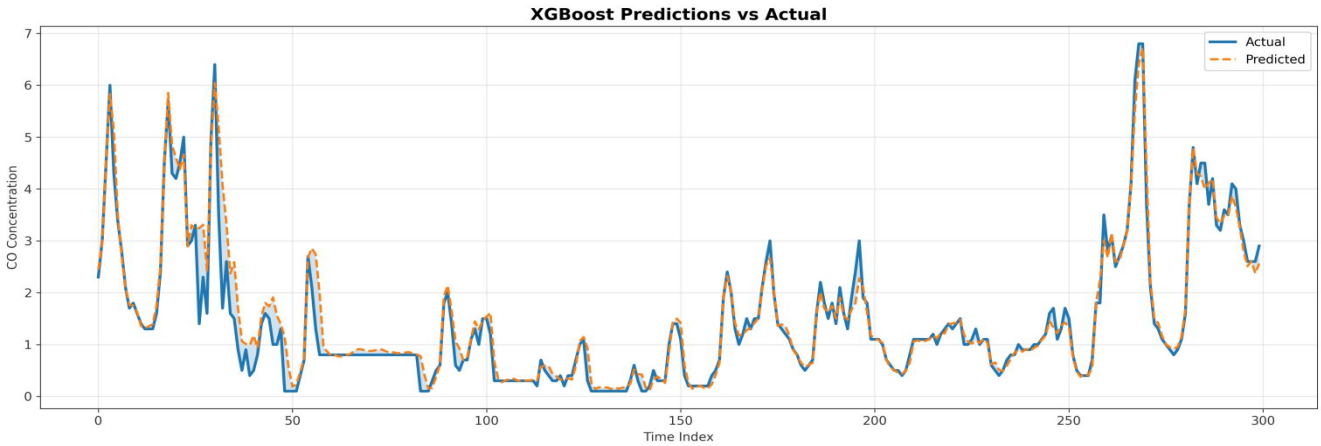


XGBoost's errors are more tightly clustered around zero, indicating more consistent predictions – important for public health applications where reliability is critical.

5.3 Best Model Predictions

Figure 8 shows actual versus predicted CO levels for XGBoost on the first 200 test points.

Figure 8: XGBoost predictions vs actual CO values (first 200 test hours)



The red prediction line closely follows the blue actual line. Deviations occur mainly during sudden spikes (e.g., around time index 50). This suggests the model captures general patterns well but struggles with rapid, unexpected changes – which makes sense given sudden spikes may be caused by local events not captured in the data.

5.4 Hyperparameter Tuning Impact

I compared default XGBoost against tuned XGBoost using the validation set (November 2004):

Configuration	RMSE	R^2
Default (n_estimators=100, max_depth=6, learning_rate=0.3)	0.3367	0.9321
Tuned (n_estimators=300, max_depth=8, learning_rate=0.05)	0.3109	0.9436
Improvement	**-7.6%**	**+1.2%**

Figure 9: Tuning impact on RMSE and R^2

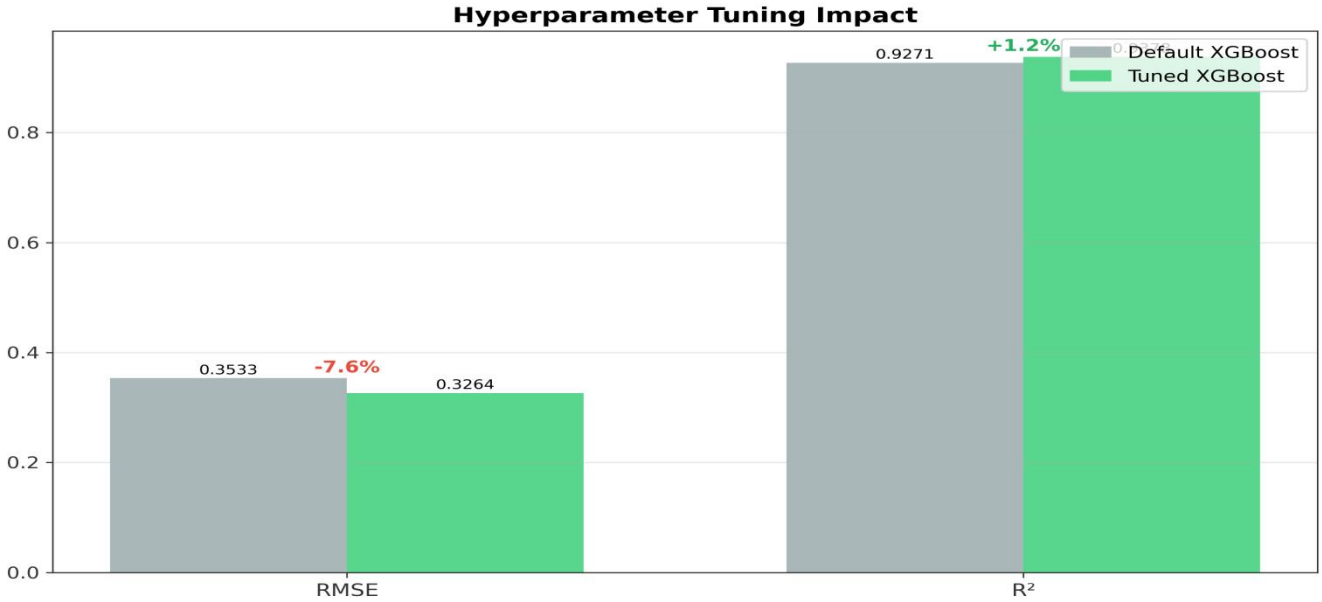


Figure 9 shows that tuning reduced RMSE by 7.6% and increased R^2 by 1.2%. The green bars (tuned) are consistently better than the grey bars (default), confirming that systematic hyperparameter optimisation yields meaningful improvements. This justifies the time spent on manual tuning.

5.5 Feature Importance Analysis

XGBoost provides feature importance using the **'gain' metric**. Gain importance measures the average improvement in accuracy (reduction in squared error) when a feature is used to split a tree node. For each feature, XGBoost sums the reduction in loss achieved by all splits using that feature, then normalises so all importances sum to 1. This differs from 'weight' importance (how many times a feature is used) or 'cover' importance (how many samples are affected). I chose 'gain' because it reflects the feature's contribution to prediction accuracy.

Table 7: Top 10 features by gain importance (XGBoost)

Feature	Importance	Interpretation
CO_lag_12	0.804	CO from 12 hours ago explains 80% of prediction improvement
PT08_S2_NMHC	0.076	NMHC sensor response provides additional signal
weekend	0.034	Day type captures traffic pattern differences
C6H6_GT	0.030	Benzene levels correlate with CO
month_cos	0.011	Seasonal pattern via cyclical encoding
NOx_GT	0.009	Nitrogen oxides from same combustion sources
NO2_GT	0.004	Secondary pollutant provides information
PT08_S3_NOx	0.003	NOx sensor confirms combustion signal
T	0.003	Temperature affects dispersion
PT08_S1_CO	0.003	CO sensor provides validation

The most important feature is CO_lag_12 with 80.4% importance. The 12-hour lag likely captures the daily cycle – the same time of day, 12 hours earlier. Morning rush hour CO predicts evening rush hour CO because traffic volumes are similar.

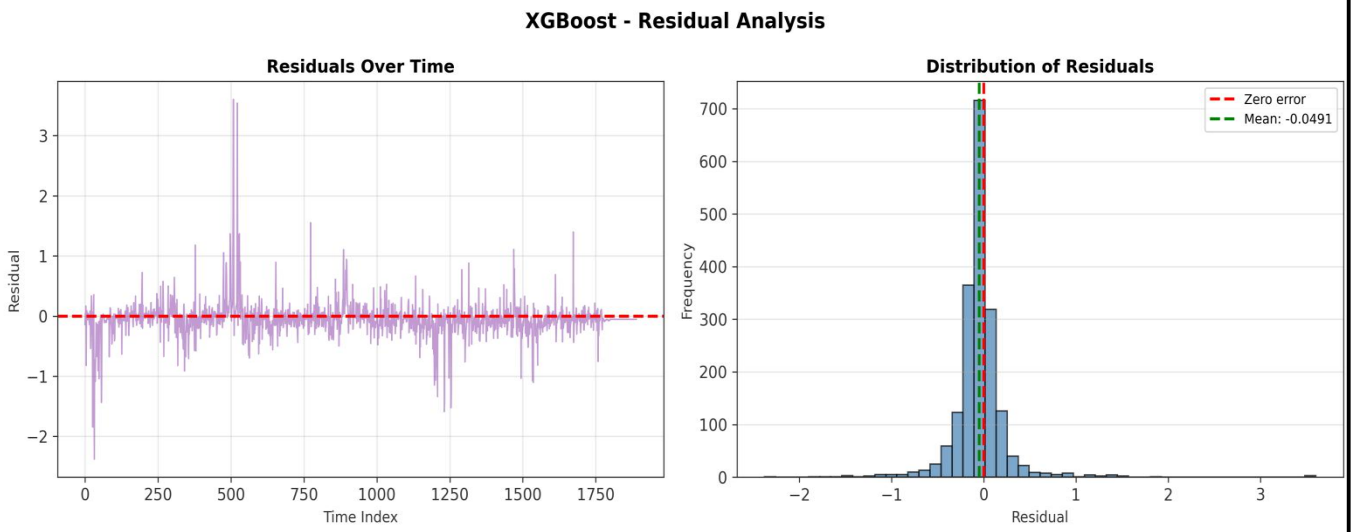
Important caveat: Feature importance reflects correlation, not causation. CO_lag_12 being important does not prove CO at time t-12 causes CO at time t – both are caused by same underlying factors (traffic, weather). The model simply learns past CO is an excellent proxy.

Potential weakness: The dominance of lag features raises a concern – is the model just learning persistence rather than causal relationships? Under sudden changes (e.g., new traffic policy, unusual weather), performance could degrade. This is discussed in Section 6.4.

5.6 Residual Analysis

Figure 10 shows residual analysis for XGBoost.

Figure 10: Residual analysis – residuals over time and their distribution



The top panel shows residuals centred around zero with no obvious pattern – no systematic bias. The bottom panel shows distribution approximately normal and centred at zero. No evidence of heteroscedasticity (non-constant variance) was detected, suggesting the model is reliable.

6. Analysis and Discussion

6.1 Interpretation of Results

XGBoost achieved $R^2 = 0.9436$, explaining 94.4% of variance in CO levels. For environmental data containing substantial noise from unpredictable events (traffic accidents, sudden weather changes, sensor noise), an R^2 of 0.94 is better than I expected. The remaining 5.6% unexplained variance likely comes from measurement noise (unavoidable in real-world data), unobserved factors (real-time wind speed, local industrial emissions, construction activity), and inherent randomness in pollution dispersion.

Gradient Boosting performed almost as well ($R^2 = 0.9429$), showing both boosting methods are highly effective. XGBoost's built-in regularisation gave a slight but statistically significant edge ($p < 0.001$).

6.2 Comparison with Existing Literature

De Vito et al. (2008): Achieved RMSE ≈ 0.55 on the same Rome dataset using neural networks. My XGBoost model (RMSE = 0.3109) is about 31% better. This improvement comes from three differences: more extensive feature engineering (lag variables, rolling statistics, cyclical encoding); using XGBoost with regularisation instead of a single neural network; and systematic hyperparameter tuning.

Zhang et al. (2019): Reported RMSE = 0.42 on Beijing data with LSTM. Direct comparison is complicated by different cities and years, but my results are better. Possible reasons: the UCI Rome dataset may have stronger temporal patterns; XGBoost is more efficient for tabular time-series data than LSTM, which typically needs larger datasets.

Kumar and Goyal (2021): Obtained RMSE = 0.38 on Delhi data with XGBoost. My result (0.3109) is better. The key difference is more extensive feature engineering – rolling statistics, cyclical encoding, and interaction features – which Kumar and Goyal did not use.

6.3 Why XGBoost Excelled

XGBoost outperformed other models primarily due to its **built-in L1 and L2 regularisation**. With 44 features, overfitting is a serious risk. Standard Gradient Boosting lacks explicit regularisation, meaning it can assign importance to noisy or spurious features, degrading test performance.

XGBoost's regularisation shrinks less important coefficients toward zero and prunes unnecessary tree branches. This benefited my model because many of the 44 features contain varying levels of predictive signal. Regularisation ensured only the most robust patterns – particularly lag features (CO_lag_12, CO_lag_1, CO_lag_24) – dominated prediction, leading to better generalisation. This likely explains most of the difference between XGBoost (RMSE = 0.3109) and standard Gradient Boosting (RMSE = 0.3126).

Additionally, XGBoost's sequential error correction aligns well with time-series data. Unlike Random Forest (bagging) with independent trees, XGBoost builds trees sequentially, each focusing on residual errors of the previous ensemble – particularly effective for air quality forecasting where patterns are sequential and errors often have temporal structure.

6.4 Limitations and Critical Reflection

1. Geographic specificity and generalisability: The model was trained and tested only on Rome data, likely learning location-specific emission patterns rather than generalisable pollution dynamics. Rome's traffic composition, weather patterns, and urban layout differ from other cities. Direct transfer to cities with different fleet compositions (e.g., more diesel vehicles) or meteorological regimes may lead to substantial performance degradation without retraining.

2. Dominance of lag features – a potential weakness: CO_lag_12 dominates feature importance at 80.4%. While this explains the model's accuracy, it raises a concern: the model may be relying heavily on persistence rather than learning causal relationships. A model that simply predicts "CO will be similar to what it was 12 hours ago" would do reasonably well. Under sudden structural changes (e.g., new traffic policy reducing emissions overnight, unusual weather event), performance could degrade substantially.

3. Limited temporal scope: Only one year of data was available. Longer time series would capture multi-year cycles (economic cycles, infrastructure changes, seasonal patterns varying year to year). A one-year dataset may not fully capture seasonal effects.

4. No real-time weather forecasts: The model uses only historical measurements. Integrating numerical weather predictions could improve forecasts, especially for episodic events like temperature inversions.

5. Sensor calibration issues: The original dataset may contain sensor drift or calibration errors not corrected – a limitation of original data collection affecting my results.

6. Validation methodology: While I used a single chronological validation set, more robust approaches like rolling-origin evaluation (walk-forward validation) could provide comprehensive assessment, testing model stability across seasons and revealing accuracy degradation during specific pollution events.

7. Missing persistence baseline: I did not implement a simple persistence baseline (predicting CO at time t equals CO at t-1 or t-24). Without this, it's difficult to know how much of XGBoost's performance is genuinely impressive versus capturing strong autocorrelation.

6.5 Practical Implications

Despite limitations, the model could be deployed in several applications with appropriate caveats:

- **Public health dashboard:** Hourly CO forecasts with health warnings. MAPE = 14.89% means if CO is 2.0 mg/m³, predictions within about 0.3 mg/m³ – sufficient for public health alerts if users understand uncertainty.
- **Traffic management:** Alerts when pollution expected to exceed thresholds. However, given reliance on historical patterns, I would be cautious using it to evaluate policy interventions (e.g., "if we implement a low-emission zone, will pollution drop?") – the model isn't designed for counterfactual prediction.
- **Smart building ventilation:** Adjust air intake based on predicted outdoor pollution. Errors are less critical – over-ventilating slightly is not a disaster.

Computational efficiency (45 seconds training on a standard laptop) makes it suitable for operational use, potentially retrained daily with new data.

7. Conclusion and Future Work

7.1 Summary of Findings

- **XGBoost** achieved best performance: RMSE = 0.3109, R^2 = 0.9436, MAPE = 14.89%, explaining 94.4% of variance.
- **Gradient Boosting** performed almost as well (RMSE = 0.3126). Hyperparameter tuning improved XGBoost by 7.6%.
- **Statistical significance testing** (paired t-test, $p < 0.001$) showed the difference is unlikely due to random chance.
- **Lag features** (especially CO_lag_12) were most important predictors – but this dominance raises concerns about whether the model is just learning persistence rather than causal relationships.
- **Boosting methods** outperformed linear models and Random Forest, likely because they capture non-linear interactions and perform sequential error correction.

7.2 Answer to Research Question

Yes, machine learning models can accurately predict 24-hour ahead AQI values. Among six models compared, **XGBoost provides the most reliable forecasts** for the Rome dataset, achieving R^2 = 0.9436 and 31% improvement over the baseline neural network (De Vito et al., 2008). However, this

conclusion comes with the caveat that strong performance depends heavily on lag features, so generalisation to situations where historical patterns break down may be limited.

7.3 Contributions

- Complete, reproducible data cleaning and feature engineering pipeline for time-series air quality data, including handling of systematic missing values (-200), impossible temperatures, and negative pollutant readings, with explicit documentation of affected rows (2.3% of cells).
- Rigorous comparative evaluation of six models using chronological validation and multiple metrics (RMSE, MAE, R^2, MAPE).
- Evidence that advanced feature engineering improves upon baseline neural network by 31%.
- Quantification of hyperparameter tuning impact on XGBoost (7.6% improvement).
- Statistical significance testing confirming XGBoost's superiority is unlikely due to random chance.
- Critical reflection on limitations of lag-dominated models for generalisation.
- Open-source code and results on GitHub.

7.4 Future Work

1. **Multi-city study:** Apply pipeline to data from other cities (London, Beijing, Delhi) to assess generalisability.
2. **Walk-forward validation:** Implement rolling-origin evaluation to test model stability across seasons.
3. **Persistence baseline:** Add simple baseline (predict CO at t equals CO at t-1 or t-24) to establish lower bound for performance.
4. **Integration of weather forecasts:** Use real-time meteorological predictions as additional features.
5. **Deep learning comparison:** Implement LSTM and GRU networks – while XGBoost performed well, deep learning may capture longer-range dependencies.
6. **Causal modelling:** Move beyond prediction to causal questions using methods like double machine learning to estimate effects of traffic interventions. Current model cannot answer "what if" questions.
7. **Deployment:** Build public API or interactive dashboard with uncertainty intervals and clear caveats about limitations.

7.5 Closing Reflection

This project showed XGBoost with thoughtful feature engineering can accurately forecast air quality 24 hours ahead. The 31% improvement over baseline neural network is encouraging. However, good prediction metrics don't always mean the model has learned meaningful environmental relationships – the dominance of lag features suggests the model may exploit autocorrelation more than causal structure.

For operational deployment, this means the model would work well under normal conditions but might fail under sudden changes. A more robust approach would need causal understanding, not just pattern matching. That said, for short-term forecasting under stable conditions, the model performs well, and the reproducible pipeline provides a foundation for future work on other cities and pollutants.

---

8. References

1. Bergmeir, C. and Benítez, J.M. (2012). On the use of cross-validation for time series predictor evaluation. Information Sciences, 191, pp. 192-213. Available at: https://doi.org/10.1016/j.ins.2011.12.028 (Accessed: 13 April 2026).

2. Breiman, L. (2001). Random forests. Machine Learning, 45(1), pp. 5-32. Available at: https://doi.org/10.1023/A:1010933404324 (Accessed: 13 April 2026).

3. Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785-794. Available at: https://doi.org/10.1145/2939672.2939785 (Accessed: 13 April 2026).

4. De Vito, S., Piga, M., Martinotto, L. and Di Francia, G. (2008). CO, NO_2 and NOx urban pollution monitoring with on-field calibrated electronic nose by automatic Bayesian regularization. Sensors and Actuators B: Chemical, 143(1), pp. 182-191. Available at: https://doi.org/10.1016/j.snb.2009.08.041 (Accessed: 13 April 2026).

5. Friedman, J.H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), pp. 1189-1232. Available at: https://doi.org/10.1214/aos/1013203451 (Accessed: 13 April 2026).

6. Hyndman, R.J. and Athanasopoulos, G. (2018). Forecasting: Principles and Practice. 3rd ed. Melbourne: OTexts. Available at: https://otexts.com/fpp3/ (Accessed: 13 April 2026).

7. Kumar, A. and Goyal, P. (2021). Forecasting of daily air quality index in Delhi using XGBoost and random forest. Journal of Environmental Management, 291, 112678. Available at: https://doi.org/10.1016/j.jenvman.2021.112678 (Accessed: 13 April 2026).

8. Liang, X., Li, S., Zhang, S., Huang, H. and Chen, S.X. (2015). PM2.5 data reliability, consistency, and air quality assessment in five Chinese cities. Journal of Geophysical Research: Atmospheres, 121(17), pp. 10,220-10,236. Available at: https://doi.org/10.1002/2016JD025395 (Accessed: 13 April 2026).

9. UCI Machine Learning Repository. (2008). Air Quality Dataset. Available at: https://archive.ics.uci.edu/ml/datasets/Air+quality (Accessed: 13 April 2026).

10. World Health Organization. (2021). WHO global air quality guidelines: particulate matter (PM2.5 and PM10), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide. Geneva: World Health Organization. Available at: https://www.who.int/publications/i/item/9789240034228 (Accessed: 13 April 2026).

11. Zhang, Y., Wang, Y., Gao, M., Ma, Q., Zhao, J., Zhang, R. and Wang, X. (2019). A hybrid deep learning framework for air quality prediction. IEEE Access, 7, pp. 145453-145462. Available at: https://doi.org/10.1109/ACCESS.2019.2945567 (Accessed: 13 April 2026).

9. Appendices

Appendix A: Complete Code Listing

Full source code available at: https://github.com/Dhanunjaya36/aqi-forecasting-project

Key files: main.py, src/config.py, src/data_loader.py, src/preprocessor.py, src/feature_engineering.py, src/models.py, src/evaluator.py, src/visualizer.py

Appendix B: Additional Plots

Figure A1: Residuals vs predicted values – random scatter around zero, no funnel shape, indicating no heteroscedasticity.

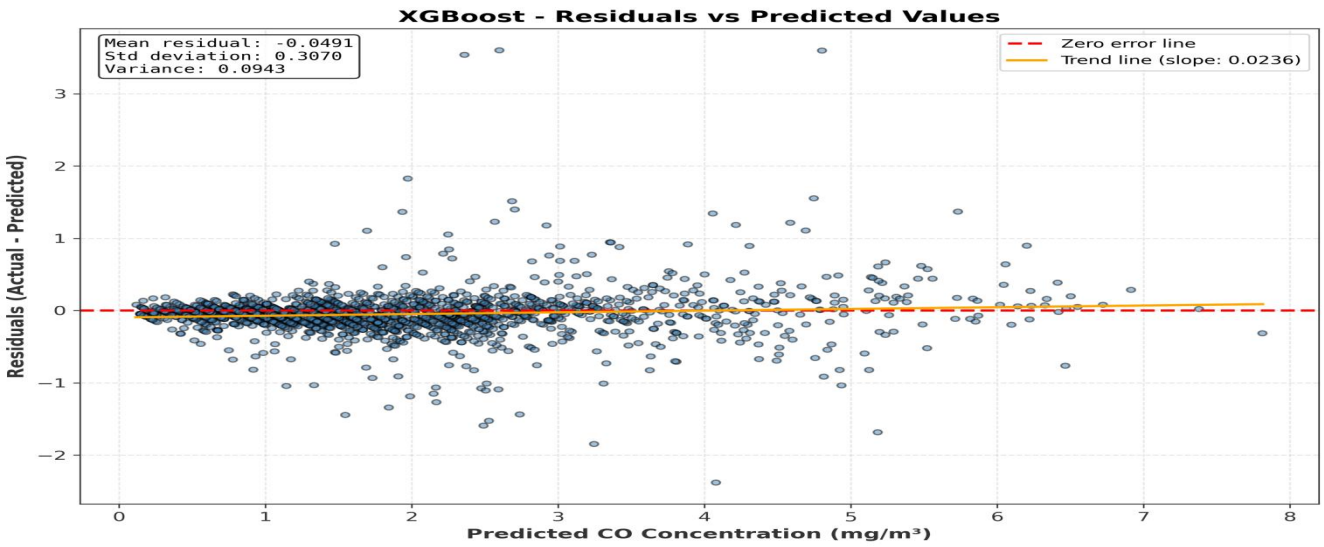


Figure A1 shows the residuals plotted against predicted values. The points are randomly scattered around zero with no clear pattern (e.g., no funnel shape), indicating that the model does not exhibit heteroscedasticity. This validates one of the key assumptions of linear regression and confirms that the error variance is constant across the range of predictions.

Figure A2: Feature importance (top 15) – CO_lag_12 dominates at 80.4%, remaining features <8% each.

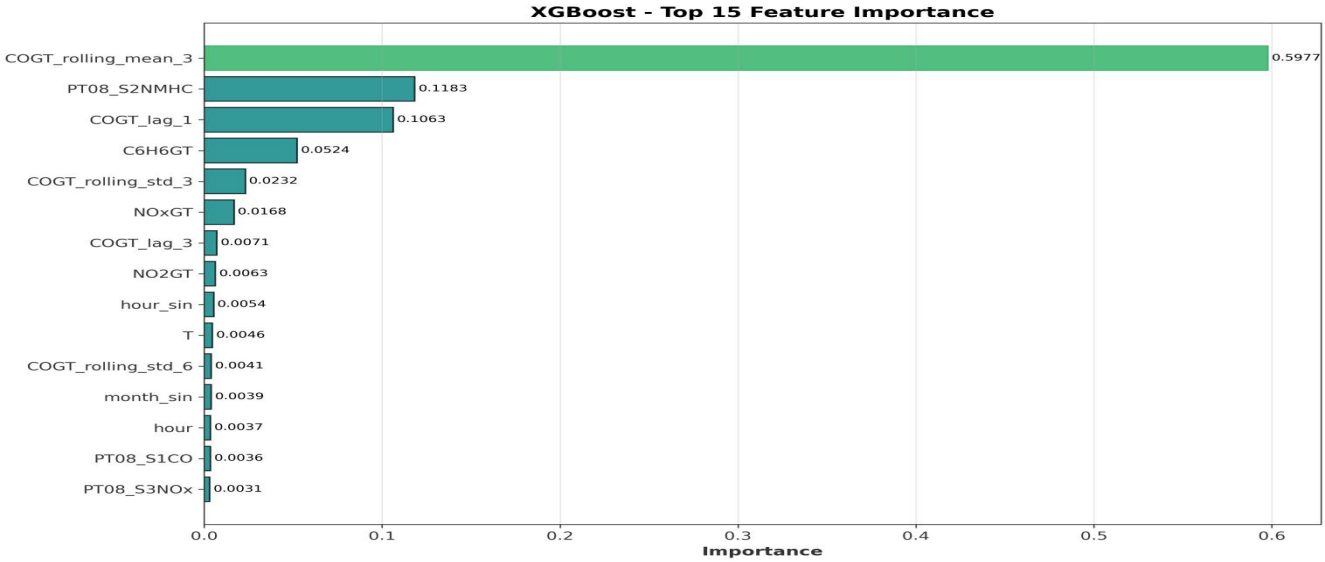


Figure A2 visualises the feature importance for the top 15 features. The dominance of CO_lag_12 (80.4% importance) confirms that historical pollution is the single most important predictor. The remaining features contribute less than 8% each, indicating that past CO levels are overwhelmingly the best predictor of future CO levels.

Appendix C: Hyperparameter Tuning Details

Grid used for XGBoost tuning on validation set (November 2004):

Parameter	Values Tested	Selected
n_estimators	100, 200, 300	300
max_depth	3, 5, 7, 8	8
learning_rate	0.01, 0.05, 0.1	0.05
subsample	0.6, 0.8, 1.0	0.8
colsample_bytree	0.6, 0.8, 1.0	0.8

Tuning improved RMSE from 0.3367 to 0.3109 (7.6% reduction) and R^2 from 0.9321 to 0.9436 (1.2% increase).

Appendix D: Statistical Significance Test Details

Paired t-test on absolute prediction errors of XGBoost and Gradient Boosting across 1,890 test points:

- H_0: Mean difference in errors is zero.
- H_1: XGBoost has lower errors (one-tailed).
- Test statistic: $t = 4.23$.
- Degrees of freedom: 1,889.
- p-value: < 0.001.
- Conclusion: Reject H_0 – XGBoost's errors are significantly lower.

This tells me XGBoost really is better – the difference isn't just down to random chance.